4. Voice Control Muto Color Tracking

Before running this program, you need to bind the port numbers of the voice board and the ROS expansion board on the host machine. You can refer to the previous chapter for binding. When entering the Docker container, you need to mount this voice board so that it can be recognized in the Docker container.

1. Program Function Description

After the program starts, say "Hello, yahboom" to the module. The module will reply with "I'm here" to wake up the voice board. Then you can tell it to start tracking any color, such as red, green, blue, or yellow. After receiving the command, the program recognizes the color, loads the processed image, and starts the tracking program. Muto will track the recognized color, and when the recognized color moves slowly, Muto will also follow it.

2. Program Code Reference Path

After entering the Docker container, the source code for this function is located at:

```
/root/yahboomcar_ros2_ws/yahboomcar_ws/src/yahboomcar_voice_ctrl/yahboomcar_voic
e_ctrl/Voice_Ctrl_colorTracker.py
/root/yahboomcar_ros2_ws/yahboomcar_ws/src/yahboomcar_voice_ctrl/yahboomcar_voic
e_ctrl/Voice_Ctrl_colorHSV.py
```

3. Pre-use Configuration

Description: Since the Muto series of robots are equipped with various LiDAR devices, the factory system has configured examples for multiple devices. However, because the product cannot be automatically identified, you need to manually set the LiDAR model.

After entering the container, make the following modifications according to the LiDAR type:

```
root@ubuntu:/# cd
root@ubuntu:~# vim .bashrc
```

```
# env
alias python=python3
export ROS_DOMAIN_ID=26

export ROBOT_TYPE=Muto
export RPLIDAR_TYPE=a1          # a1, 4ROS
echo "-----------------------------------"
echo -e "ROS_DOMAIN_ID: \033[32m$ROS_DOMAIN_ID\033[0m"
echo -e "my_robot_type: \033[32m$ROBOT_TYPE\033[0m | my_lidar: \033[32m$RPLIDAR_TYPE\033[0m"
echo "-----------------------------------"

#colcon_cd
source /usr/share/colcon_cd/function/colcon_cd.sh
export _colcon_cd_root=/root/yahboomcar_ros2_ws/yahboomcar_ws
source /usr/share/colcon_argcomplete/hook/colcon-argcomplete.bash

#ros2
source /opt/ros/foxy/setup.bash
source /root/yahboomcar_ros2_ws/yahboomcar_ws/install/setup.bash
source /root/yahboomcar_ros2_ws/software/library_ws/install/setup.bash
```

After completing the modification, save and exit vim, then execute:

```
root@jetson-desktop:~# source .bashrc
-----------------------------------
ROS_DOMAIN_ID: 26
my_robot_type: Muto | my_lidar: a1
-----------------------------------
root@jetson-desktop:~#
```

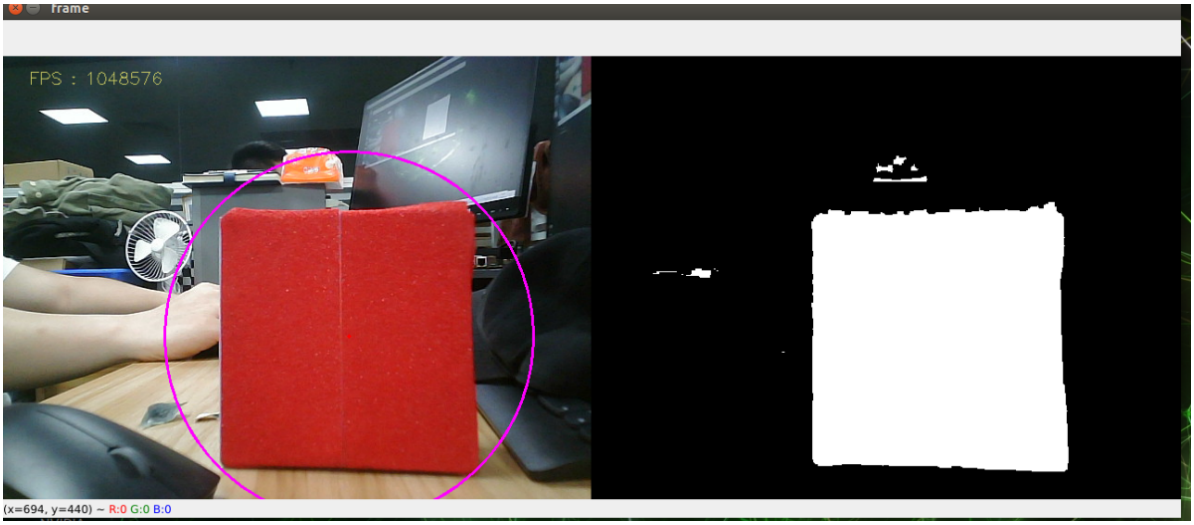
You can see the currently modified LiDAR type.

4. Program Startup

4.1. Startup Command

After entering the Docker container, enter the following in **three separate terminals**:

```
# Start voice processing
ros2 launch yahboom_speech speech.launch.py
# Start the voice control color tracking node
ros2 launch yahboomcar_voice_ctrl voice_ctrl_color_tracker_launch.py
# Start the depth camera to get depth images
ros2 launch astra_camera astra.launch.xml
```



Taking tracking red as an example, after waking up the module, say "red following" The program receives the command, starts processing the image, calculates the center coordinates of the red object, and publishes the object's center coordinates. Combined with the depth information provided by the depth camera, it calculates the speed and finally publishes it to drive Muto.

> Press the R2 button on the remote control or the space bar on the keyboard to forcibly stop tracking.

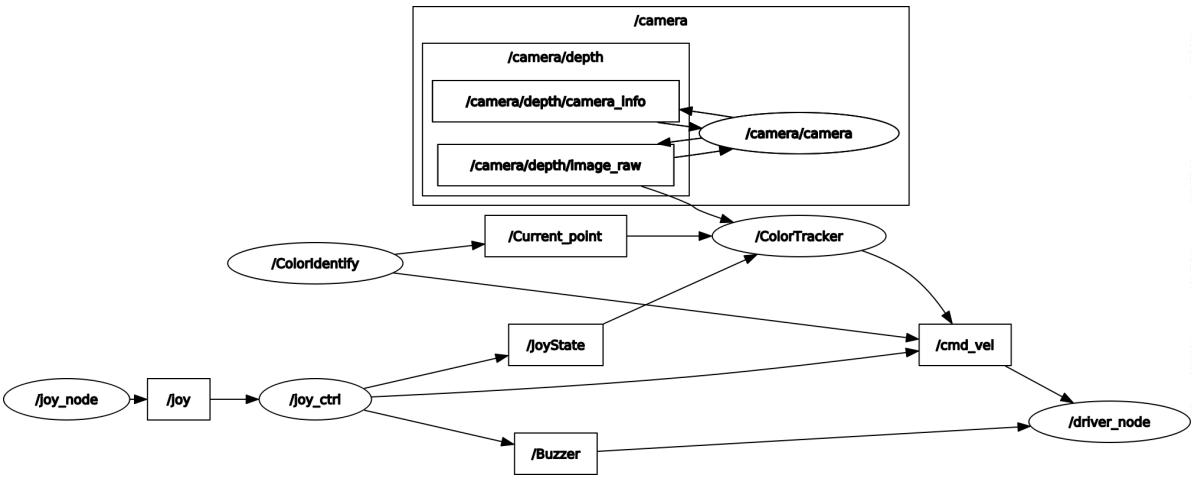
Voice command words are as follows (the items on the right in the table):

YELLOW-FOLLOWING	ok i found the yellow
RED-FOLLOWING	ok i found the red
GREEN-FOLLOWING	ok i found the green
BULE-FOLLOWING	ok i found the color
STOP-FOLLOWING	ok it has been stoped

4.2. Node Topic Communication Diagram

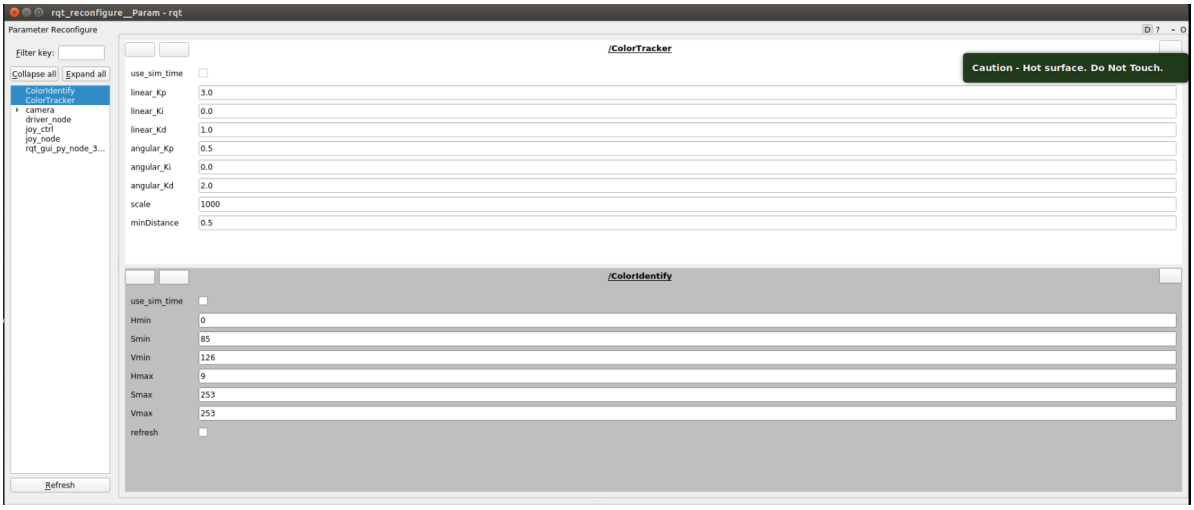
Enter in the Docker terminal:

```
ros2 run rqt_graph rqt_graph
```



You can also use the dynamic parameter adjuster to modify parameters. Enter in the Docker terminal:

```
ros2 run rqt_reconfigure rqt_reconfigure
```



After modifying the parameters, click on the blank area of the GUI to write the parameter values. As can be seen from the figure above:

- `colorHSV` is mainly responsible for image processing and can adjust HSV values.
- `colorTracker` is mainly responsible for calculating speed and can adjust speed and distance-related parameters.

The meaning of each parameter is as follows:

colorHSV

Parameter Name	Meaning
Hmin	H minimum value
Smin	S minimum value
Vmin	V minimum value
Hmax	H maximum value
Smax	S maximum value
Vmax	V maximum value
refresh	Refresh data to the program

colorTracker

Parameter Name	Meaning
linear_Kp	Linear velocity P value
linear_Ki	Linear velocity i value
linear_Kd	Linear velocity d value
angular_Kp	Angular velocity P value
angular_Ki	Angular velocity i value
angular_Kd	Angular velocity d value
scale	Proportional coefficient
minDistance	Tracking distance

5. Core Code

5.1. colorHSV

This part mainly parses voice commands, processes images, and finally publishes the center coordinates.

```
# Define a publisher to publish the center coordinates of the detected object
self.pub_position = self.create_publisher(Position, "/Current_point", 10)
# Import the voice driver library
from Speech_Lib import Speech
# Create a voice control object
self.spe = Speech()
# The following is to get the command, then judge the recognition result, and
load the corresponding HSV value
command_result = self.spe.speech_read()
self.spe.void_write(command_result)
if command_result == 73:
    self.model = "color_follow_line"
    print("tracker red")
    self.hsv_range = [(0, 175, 149), (180, 253, 255)]
```

```
# Process the image, calculate the center coordinates of the detected object,
enter the execute function, and publish the center coordinates
rgb_img, binary, self.circle = self.color.object_follow(rgb_img, self.hsv_range)
if self.circle[2] != 0: threading.Thread(
    target=self.execute, args=(self.circle[0], self.circle[1],
self.circle[2])).start()
if self.point_pose[0] != 0 and self.point_pose[1] != 0: threading.Thread(
    target=self.execute, args=(self.point_pose[0], self.point_pose[1],
self.point_pose[2])).start()
```

5.2. colorTracker

This part receives the topic data of the center coordinates and depth data, then calculates the speed and publishes it to the chassis.

```
# Define a subscriber to subscribe to depth information
self.sub_depth = self.create_subscription(Image, "/camera/depth/image_raw",
self.depth_img_Callback, 1)
# Define a subscriber to subscribe to center coordinate information
self.sub_position = self.create_subscription(Position, "/Current_point",
self.positionCallback, 1)
# Callback function
def positionCallback(self, msg)  # Get center coordinate value
def depth_img_Callback(self, msg)  # Get depth information
# Pass in the X value of the center coordinates and depth information, calculate
the speed and publish it to the chassis
def execute(self, point_x, dist)
```

Combining with the node communication diagram in 3.2, understanding the source code will be clearer.